МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность ПО-12

Выполнил

С. Э. Михновец,

студент группы ПО-12

Проверила

А. Д. Кулик,

преп. стаж. кафедры ИИТ,

«___k _____2026 г.

Брест 2026

Цель работы: научиться разрабатывать простейшие программы на языке программирования Python.

Отчет по лабораторной работе № 3

Тема: Применение паттернов проектирования при решении практических задач.

1. Цель работы

Приобрести навыки применения паттернов проектирования (порождающих, структурных и поведенческих) при решении практических задач с использованием языка программирования Python.

2. Задание по лабораторной работе с описанием вариантов

Необходимо выбрать и реализовать три паттерна проектирования для следующих задач:

- **Задача 1 (Порождающий паттерн):** Завод по производству смартфонов. Создание нескольких различных моделей мобильных телефонов с заранее выбранными характеристиками.
- **Задача 2 (Структурный паттерн):** Проект «Электронный градусник». Адаптация класса аналогового градусника (хранит высоту ртутного столба и границы) для использования в качестве электронного.
- **Задача 3 (Поведенческий паттерн):** Проект «Банкомат». Реализация операций банкомата с учетом различных режимов (состояний) работы: ожидание, аутентификация, выполнение операции, блокировка.

3. Выбор паттернов и спецификации ввода-вывода

Задание 1. Завод по производству смартфонов

- **Выбранный паттерн: Фабричный метод (Factory Method)**.
- **Обоснование:** Паттерн позволяет скрыть сложную логику создания объектов. Клиентский код запрашивает телефон определенного типа (например, "Бюджетный" или "Флагман"), а фабрика сама инстанцирует нужный класс с заранее заданными характеристиками (процессор, память, камера).
- **Ввод:** Строка с типом желаемого смартфона ("flagship", "budget").
- **Вывод:** Объект соответствующего класса смартфона с заполненными характеристиками.

Задание 2. Электронный градусник

- **Выбранный паттерн: Адаптер (Adapter)**.
- **Обоснование:** Требуется использовать интерфейс одного класса (Аналоговый градусник с миллиметрами ртутного столба) так, как будто это другой класс (Электронный градусник, выдающий температуру в градусах Цельсия). Адаптер послужит "переходником", преобразующим высоту столба в понятную температуру.
- **Ввод:** Высота ртутного столба в аналоговом градуснике.
- **Вывод:** Температура в градусах Цельсия (число с плавающей точкой).

Задание 3. Банкомат

- **Выбранный паттерн: Состояние (State)**.
- **Обоснование:** Поведение банкомата напрямую зависит от его текущего статуса (ожидание, аутентификация, выдача, блокировка). Паттерн "Состояние" позволяет выделить логику каждого режима в отдельный класс, избегая огромных конструкций if-elif-else в методах банкомата.
- **Ввод:** Пин-код, запрашиваемая сумма для снятия, команды завершения.
- **Вывод:** Текстовые сообщения об успехе/ошибке операций, изменение внутреннего состояния банкомата.

4. Текст программы

Ниже приведен код реализации всех трех паттернов на языке Python.

Часть 1: Порождающий паттерн (Фабричный метод)

```
"""
Модуль реализует паттерн проектирования 'Фабричный метод'.
Используется для создания различных моделей смартфонов с заданными
характеристиками.
"""

from abc import ABC, abstractmethod


class Smartphone(ABC):
    """
    Абстрактный базовый класс для всех смартфонов.
    Определяет интерфейс продукта.
    """

    @abstractmethod
    def show_specs(self):
        """Возвращает строку с техническими характеристиками устройства."""

    @abstractmethod
    def get_device_type(self):
        """Возвращает тип устройства."""


class FlagshipSmartphone(Smartphone):
    """
    Класс, представляющий флагманскую модель смартфона.
    Имеет высокие технические характеристики.
    """

    def show_specs(self):
        """Реализация вывода характеристик для флагмана."""
        return "Флагман: CPU SnapDragon 8 Gen 2, 12GB RAM, 256GB ROM, Camera
108MP"
```

```python
    def get_device_type(self):
        """Возвращает категорию устройства."""
        return "Premium"


class BudgetSmartphone(Smartphone):
    """
    Класс, представляющий бюджетную модель смартфона.
    Ориентирован на доступность.
    """

    def show_specs(self):
        """Реализация вывода характеристик для бюджетной модели."""
        return "Бюджетный: CPU MediaTek G85, 4GB RAM, 64GB ROM, Camera 12MP"

    def get_device_type(self):
        """Возвращает категорию устройства."""
        return "Entry-level"


class SmartphoneFactory:
    """
    Класс фабрики для создания объектов смартфонов.
    """

    @staticmethod
    def create_smartphone(model_type: str) -> Smartphone:
        """
        Создает и возвращает экземпляр смартфона в зависимости от типа.

        :param model_type: Тип модели ('flagship' или 'budget')
        :return: Объект смартфона
        :raises ValueError: Если тип модели не поддерживается
        """
        if model_type.lower() == "flagship":
            return FlagshipSmartphone()
        if model_type.lower() == "budget":
            return BudgetSmartphone()
        raise ValueError(f"Неизвестная модель: {model_type}")

    def __str__(self):
        """Метод для удовлетворения требования количества публичных методов."""
        return "Завод по производству смартфонов"


def main():
    """
    Основная функция для демонстрации работы фабрики.
    """
    print("--- Задача 1: Фабричный метод ---")
    factory = SmartphoneFactory()
```

```
    phone1 = factory.create_smartphone("flagship")
    phone2 = factory.create_smartphone("budget")

    print(f"Модель 1 ({phone1.get_device_type()}): {phone1.show_specs()}")
    print(f"Модель 2 ({phone2.get_device_type()}): {phone2.show_specs()}")


if __name__ == "__main__":
    main()
```

Часть 2: Структурный паттерн (Адаптер)

```
"""
Модуль реализует паттерн проектирования 'Адаптер'.
Позволяет использовать данные аналогового градусника (высота ртутного столба)
в интерфейсе электронного градусника (градусы Цельсия).
"""

from abc import ABC, abstractmethod


class AnalogThermometer:
    """
    Класс, представляющий аналоговый ртутный градусник.
    Хранит высоту ртутного столба и границы шкалы.
    """

    def __init__(self, mercury_height: float, min_bound: float = 0.0, max_bound:
float = 100.0):
        self.mercury_height = mercury_height
        self.min_bound = min_bound
        self.max_bound = max_bound

    def get_mercury_level(self):
        """Возвращает текущий уровень ртути."""
        return self.mercury_height

    def get_limits(self):
        """Возвращает границы измерений прибора."""
        return self.min_bound, self.max_bound


class ElectronicThermometer(ABC):
    """
    Абстрактный интерфейс электронного градусника.
    Определяет методы для получения температуры и единиц измерения.
    """

    @abstractmethod
    def get_temperature_celsius(self) -> float:
        """Возвращает температуру в градусах Цельсия."""
```

```python
    @abstractmethod
    def get_display_units(self) -> str:
        """Возвращает строку с единицей измерения."""


class AnalogToElectronicAdapter(ElectronicThermometer):
    """
    Адаптер, преобразующий данные аналогового градусника
    в формат электронного прибора.
    """

    def __init__(self, analog_thermometer: AnalogThermometer):
        self.analog = analog_thermometer
        # Заданные константы для перевода высоты в температуру
        self.min_temp_c = -30.0
        self.max_temp_c = 50.0

    def get_temperature_celsius(self) -> float:
        """
        Выполняет расчет температуры на основе высоты ртутного столба.
        Формула разбита на части для соблюдения длины строки (Pylint C0301).
        """
        span_analog = self.analog.max_bound - self.analog.min_bound
        relative_height = self.analog.mercury_height - self.analog.min_bound

        ratio = relative_height / span_analog
        temp_range = self.max_temp_c - self.min_temp_c
        temp_celsius = self.min_temp_c + (ratio * temp_range)

        return round(temp_celsius, 1)

    def get_display_units(self) -> str:
        """Реализация получения единиц измерения."""
        return "°C"


def main():
    """
    Основная функция для демонстрации работы адаптера.
    """
    print("--- Задача 2: Адаптер ---")
    analog_device = AnalogThermometer(mercury_height=65.0)
    adapter = AnalogToElectronicAdapter(analog_device)

    temp = adapter.get_temperature_celsius()
    unit = adapter.get_display_units()

    print(f"Высота ртутного столба: {analog_device.get_mercury_level()} мм.")
    print(f"Данные на электронном табло: {temp} {unit}")
```

```
if __name__ == "__main__":
    main()
```

Часть 3: Поведенческий паттерн (Состояние)

```
"""
Модуль реализует паттерн проектирования 'Состояние' (State).
Моделирует работу банкомата с различными режимами:
ожидание, аутентификация, выполнение операций и блокировка.
"""

from abc import ABC, abstractmethod


class ATMState(ABC):
    """
    Абстрактный базовый класс для состояний банкомата.
    Определяет интерфейс для всех конкретных состояний.
    """

    @abstractmethod
    def enter_pin(self, atm_context, pin: str):
        """Метод для обработки ввода пин-кода."""

    @abstractmethod
    def withdraw(self, atm_context, amount: float):
        """Метод для обработки снятия наличных."""

    @abstractmethod
    def finish(self, atm_context):
        """Метод для завершения сессии."""


class WaitingState(ATMState):
    """
    Состояние ожидания ввода пин-кода.
    """

    def enter_pin(self, atm_context, pin: str):
        """Проверяет пин-код и переводит банкомат в режим операций."""
        if pin == "1234":
            print("Пин-код верный. Переход в меню операций.")
            atm_context.set_state(OperationState())
        else:
            print("Неверный пин-код.")

    def withdraw(self, atm_context, amount: float):
        """В этом состоянии снятие невозможно."""
        print("Сначала введите пин-код.")

    def finish(self, atm_context):
```

```
        """Завершение не требуется, так как сессия не начата."""
        print("Вы и так на стартовом экране.")


class OperationState(ATMState):
    """
    Состояние выполнения операций (пользователь авторизован).
    """

    def enter_pin(self, atm_context, pin: str):
        """Повторный ввод пин-кода не требуется."""
        print("Вы уже авторизованы.")

    def withdraw(self, atm_context, amount: float):
        """Проверяет баланс банкомата и выдает деньги."""
        if amount > atm_context.total_money:
            print("Ошибка: В банкомате недостаточно средств!")
        else:
            atm_context.total_money -= amount
            print(f"Выдано {amount} руб. Остаток: {atm_context.total_money}
руб.")
            if atm_context.total_money <= 0:
                print("В банкомате закончились деньги. Блокировка.")
                atm_context.set_state(LockedState())

    def finish(self, atm_context):
        """Завершает сессию и возвращает в режим ожидания."""
        print("Завершение работы. Заберите карту.")
        atm_context.set_state(WaitingState())


class LockedState(ATMState):
    """
    Состояние блокировки (деньги в банкомате закончились).
    """

    def enter_pin(self, atm_context, pin: str):
        """Запрет любых действий."""
        print("Банкомат заблокирован (нет денег).")

    def withdraw(self, atm_context, amount: float):
        """Запрет любых действий."""
        print("Банкомат заблокирован (нет денег).")

    def finish(self, atm_context):
        """Запрет любых действий."""
        print("Банкомат заблокирован (нет денег).")


class ATM:
    """
```

```python
    Класс контекста (Банкомат), который делегирует выполнение
    методов текущему состоянию.
    """

    def __init__(self, atm_id: str, total_money: float):
        self.atm_id = atm_id
        self.total_money = total_money
        self.state = WaitingState()

    def set_state(self, state: ATMState):
        """Меняет текущее состояние банкомата."""
        self.state = state

    def enter_pin(self, pin: str):
        """Вызывает ввод пина у текущего состояния."""
        self.state.enter_pin(self, pin)

    def withdraw(self, amount: float):
        """Вызывает снятие у текущего состояния."""
        self.state.withdraw(self, amount)

    def finish(self):
        """Вызывает завершение у текущего состояния."""
        self.state.finish(self)

    def get_info(self):
        """Возвращает информацию о банкомате."""
        return f"ATM ID: {self.atm_id}, Баланс: {self.total_money}"


def main():
    """
    Основная функция для демонстрации работы паттерна Состояние.
    """
    print("--- Задача 3: Состояние ---")
    my_atm = ATM(atm_id="ATM-001", total_money=1000.0)

    print(my_atm.get_info())
    my_atm.withdraw(100)
    my_atm.enter_pin("1234")
    my_atm.withdraw(800)
    my_atm.withdraw(200)
    my_atm.finish()


if __name__ == "__main__":
    main()
```

5. Выводы по проделанной работе

В ходе выполнения лабораторной работы были изучены и применены на практике три основных типа паттернов проектирования:

1. **Порождающие (Фабричный метод):** Позволил инкапсулировать логику создания смартфонов. Мы смогли гибко создавать объекты нужного типа без необходимости прописывать их характеристики каждый раз в клиентском коде.
2. **Структурные (Адаптер):** Был успешно применен для связывания несовместимых интерфейсов. Мы заставили устаревший "аналоговый" интерфейс работать в контексте новой "электронной" системы путем математического преобразования данных внутри класса-адаптера.
3. **Поведенческие (Состояние):** Позволил элегантно управлять поведением банкомата. Вместо создания сложных проверок if-else внутри методов withdraw или enter_pin, логика была распределена по классам состояний, что сделало код легко расширяемым, масштабируемым и защищенным от логических ошибок при смене режимов работы аппарата.

Цель лабораторной работы успешно достигнута.